

HDOC DAY-16 ACTIVITY

QUESTION 1 – Menu-Driven Digit Operations with User-Defined Functions

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether its digits are in increasing order from left to right.
- Check whether its digits are in decreasing order from left to right.
- Find the largest digit and the smallest digit.
- Find the second largest digit and second smallest digit in the number.

1. Problem Understanding

- Input: A positive integer through the function corresponding to the selected menu option.
- Output: The result of the selected digit operation.
- A separate void function is used for each menu option and each function reads its own input.
- For second largest and second smallest, distinct digit values are considered.

Operation Summary

Choice	Operation	Main Logic
1	Check increasing order	Compare adjacent digits from left to right; every next digit must be greater than the previous digit.
2	Check decreasing order	Compare adjacent digits from left to right; every next digit must be smaller than the previous digit.
3	Largest and smallest digit	Extract all digits and maintain the maximum and minimum values.
4	Second largest and second smallest	Use digit frequencies to find the second distinct largest and smallest digits.
Other	Invalid choice	Display an appropriate message.

2. Standard Test Cases

Test Case	Choice	Input	Expected Output	Actual Output	Result
-----------	--------	-------	-----------------	---------------	--------

1	1	n = 12345	Digits are in increasing order	Digits are in increasing order	Pass
2	2	n = 54321	Digits are in decreasing order	Digits are in decreasing order	Pass
3	3	n = 58321	Largest = 8, Smallest = 1	Largest = 8, Smallest = 1	Pass
4	4	n = 58321	Second largest = 5, Second smallest = 2	Second largest = 5, Second smallest = 2	Pass
5	1	n = 12245	Digits are not in increasing order	Digits are not in increasing order	Pass
6	2	n = 54421	Digits are not in decreasing order	Digits are not in decreasing order	Pass

3. Algorithm

A. Common Menu Logic

1. Start the program.
2. Display the menu containing the four operations.
3. Read the user's choice.
4. Use switch-case to call the corresponding void function.
5. The selected function reads the positive integer, processes it, and displays the result.
6. If the choice is invalid, display an invalid-choice message.
7. Stop the program.

B. Case 1 – Check Increasing Order

8. Read the positive integer.
9. Extract the last digit and store it as the previous digit.
10. Remove the last digit.
11. Extract the next digit.
12. If the new digit is greater than or equal to the previous digit, the digits are not strictly increasing from left to right.
13. Otherwise, continue by making the current digit the previous digit.
14. After all digits are checked, display the appropriate result.

C. Case 2 – Check Decreasing Order

15. Read the positive integer.
16. Extract the last digit and store it as the previous digit.
17. Remove the last digit.
18. Extract the next digit.

19. If the new digit is less than or equal to the previous digit, the digits are not strictly decreasing from left to right.
20. Otherwise, continue by making the current digit the previous digit.
21. After all digits are checked, display the appropriate result.

4. Algorithm – Remaining Operations

A. Case 3 – Find Largest and Smallest Digit

1. Read the positive integer.
2. Initialize largest = 0 and smallest = 9.
3. Extract each digit using $n \% 10$.
4. Compare the digit with largest and update largest if necessary.
5. Compare the digit with smallest and update smallest if necessary.
6. Remove the last digit using $n / 10$ and repeat.
7. Display the largest and smallest digits.

B. Case 4 – Find Second Largest and Second Smallest Digit

8. Read the positive integer.
9. Initialize freq[10] to zero.
10. Extract each digit and increment its frequency.
11. Scan from 9 down to 0 to find the largest and second distinct largest digits.
12. Scan from 0 up to 9 to find the smallest and second distinct smallest digits.
13. If fewer than two distinct digits are present, display that a second largest or second smallest digit does not exist.
14. Otherwise, display the second largest and second smallest digits.

5. C Program

```
#include <stdio.h>

void checkIncreasing(void)
{
    int n, temp, digit, previous, increasing = 1;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    previous = temp % 10;
    temp = temp / 10;
    while (temp > 0)
    {
```

```
digit = temp % 10;
if (digit >= previous)
    increasing = 0;
previous = digit;
temp = temp / 10;
}
```

```
if (increasing)
    printf("Digits are in increasing order\n");
else
    printf("Digits are not in increasing order\n");
}
```

```
void checkDecreasing(void)
{
    int n, temp, digit, previous, decreasing = 1;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    previous = temp % 10;
    temp = temp / 10;

    while (temp > 0)
    {
        digit = temp % 10;
        if (digit <= previous)
            decreasing = 0;
        previous = digit;
        temp = temp / 10;
    }
```

```
if (decreasing)
    printf("Digits are in decreasing order\n");
else
```

```
    printf("Digits are not in decreasing order\n");  
}
```

```
void findLargestSmallest(void)  
{  
    int n, temp, digit, largest = 0, smallest = 9;  
    printf("Enter a positive integer: ");  
    scanf("%d", &n);  
    temp = n;  
  
    while (temp > 0)  
    {  
        digit = temp % 10;  
        if (digit > largest)  
            largest = digit;  
        if (digit < smallest)  
            smallest = digit;  
        temp = temp / 10;  
    }  
  
    printf("Largest digit = %d\n", largest);  
    printf("Smallest digit = %d\n", smallest);  
}
```

```
void secondLargestSmallest()  
{  
    int n, digit[20], count = 0;  
    int i, j, temp;  
    int largest, smallest;  
    int secondLargest = -1;  
    int secondSmallest = 10;  
  
    printf("Enter a positive integer: ");  
    scanf("%d", &n);
```

```
temp = n;
```

```
while(temp > 0)
```

```
{
```

```
    digit[count] = temp % 10;
```

```
    temp = temp / 10;
```

```
    count++;
```

```
}
```

```
largest = digit[0];
```

```
smallest = digit[0];
```

```
for(i = 1; i < count; i++)
```

```
{
```

```
    if(digit[i] > largest)
```

```
        largest = digit[i];
```

```
    if(digit[i] < smallest)
```

```
        smallest = digit[i];
```

```
}
```

```
for(i = 0; i < count; i++)
```

```
{
```

```
    if(digit[i] > secondLargest && digit[i] < largest)
```

```
        secondLargest = digit[i];
```

```
    if(digit[i] < secondSmallest && digit[i] > smallest)
```

```
        secondSmallest = digit[i];
```

```
}
```

```
if(secondLargest == -1)
```

```
    printf("Second largest digit does not exist.\n");
```

```
else
```

```

    printf("Second largest digit = %d\n", secondLargest);

    if(secondSmallest == 10)
        printf("Second smallest digit does not exist.\n");
    else
        printf("Second smallest digit = %d\n", secondSmallest);
}

int main(void)
{
    int choice;

    printf("MENU\n");
    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            checkIncreasing();
            break;
        case 2:
            checkDecreasing();
            break;
        case 3:
            findLargestSmallest();
            break;
        case 4:
            secondLargestSmallest();
            break;
    }
}

```

default:

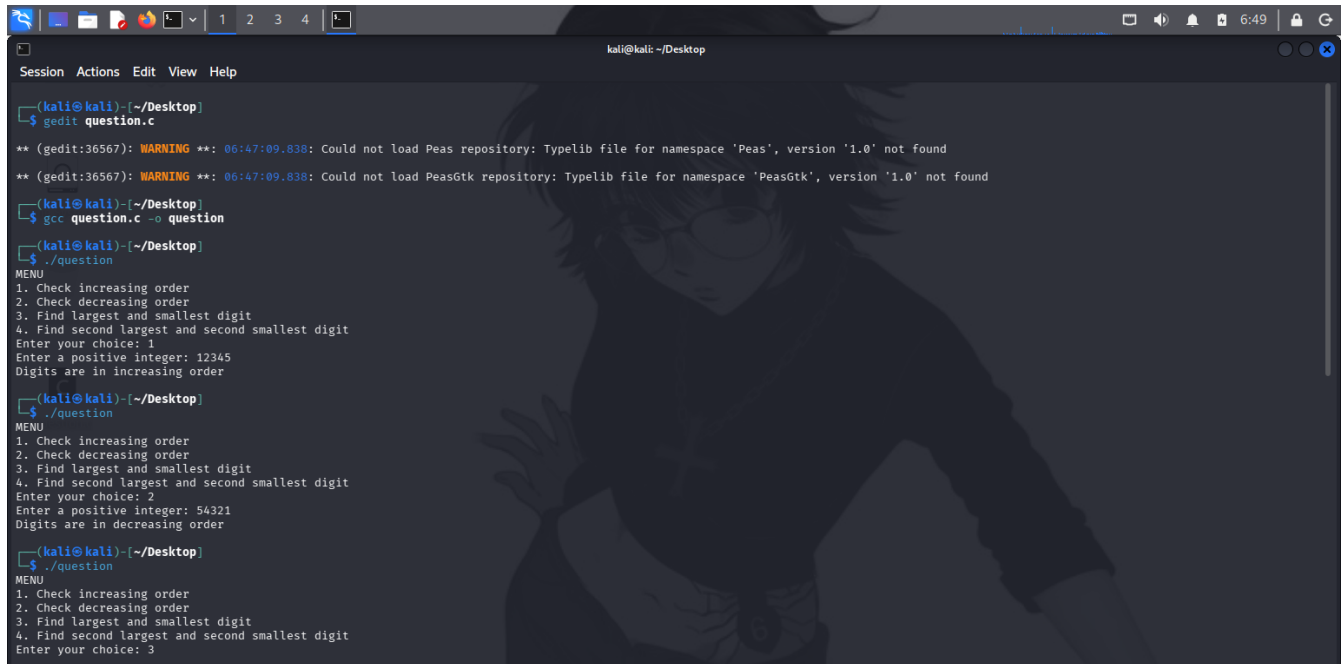
```
printf("Invalid choice.\n");
```

```
}
```

```
return 0;
```

```
}
```

6. Compilation and Execution Evidence



```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ gedit question.c

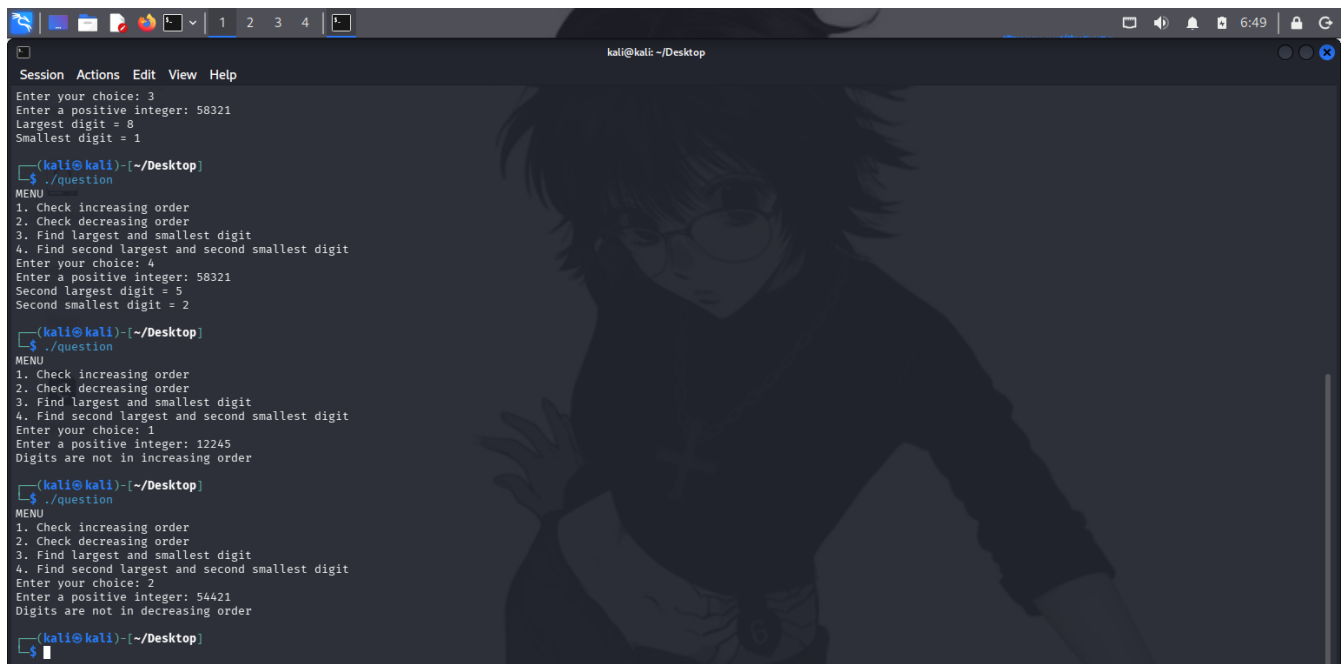
** (gedit:36567): WARNING **: 06:47:09.838: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:36567): WARNING **: 06:47:09.838: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~/Desktop]
$ gcc question.c -o question

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12345
Digits are in increasing order

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 54321
Digits are in decreasing order

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 3
```



```
kali@kali: ~/Desktop
Session Actions Edit View Help

Enter your choice: 3
Enter a positive integer: 58321
Largest digit = 8
Smallest digit = 1

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 58321
Second largest digit = 5
Second smallest digit = 2

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12245
Digits are not in increasing order

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 54421
Digits are not in decreasing order

(kali@kali)-[~/Desktop]
$
```

7. Verification of All Test Cases

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	n = 12345	Digits are in increasing order	Digits are in increasing order	Pass
2	2	n = 54321	Digits are in decreasing order	Digits are in decreasing order	Pass
3	3	n = 58321	Largest = 8, Smallest = 1	Largest = 8, Smallest = 1	Pass
4	4	n = 58321	Second largest = 5, Second smallest = 2	Second largest = 5, Second smallest = 2	Pass
5	1	n = 12245	Digits are not in increasing order	Digits are not in increasing order	Pass
6	2	n = 54421	Digits are not in decreasing order	Digits are not in decreasing order	Pass